Research Report

Predicting Polypharmacy Side Effects with Graph RAG and an LLM

Group 8

Le Van Hung Nguyen Vu Huy Nguyen Chi Dung Do Quoc An

October 2026

Contents

1 Introduction **1**
- 1.1 Problem and Motivation . . . 1
- 1.2 Related Work . . . 1
  - 1.2.1 What is already solid . . . 1
  - 1.2.2 What is still missing . . . 1
- 1.3 Objective and Project Scope . . . 2
  - 1.3.1 Objective . . . 2
  - 1.3.2 Dataset and suitability . . . 2
  - 1.3.3 Deliverable and boundaries . . . 2
- 1.4 Report Structure . . . 3

2 Background **3**
- 2.1 The Knowledge Graph . . . 3
- 2.2 Standard Graph-RAG . . . 3
- 2.3 Problem Statement . . . 4

3 Methodology **4**
- 3.1 Extending Retrieval from Two to $k \geq 3$ Drugs . . . 4
- 3.2 Pruning Before Passing to the LLM . . . 5
- 3.3 Output: Tiered, Traceable Predictions . . . 5

4 Experiments and Results **6**
- 4.1 Implementation . . . 6
  - 4.1.1 Data . . . 6
  - 4.1.2 Software Architecture . . . 7
  - 4.1.3 Subgraph Retrieval, Pruning, and Ranking . . . 7
  - 4.1.4 Large Language Model Usage . . . 8
  - 4.1.5 Grounding, Validation, and Fallback Behaviour . . . 9
- 4.2 Evaluation . . . 9
  - 4.2.1 What can and cannot be measured . . . 9
  - 4.2.2 Faithfulness to the data . . . 9
  - 4.2.3 Known interactions in larger regimens . . . 9
  - 4.2.4 Tier 3 hypotheses . . . 10
- 4.3 Limitations . . . 11

5 Conclusion and Future Work **11**

Introduction

1.1 Problem and Motivation

Polypharmacy—the concurrent use of several medications by one patient—is common among older adults and patients with chronic or complex diseases. Although often necessary, it raises the risk of drug–drug interactions (DDIs), especially polypharmacy side effects: adverse events that emerge only when two or more drugs are taken together and cannot be attributed to a single drug. Such interactions contribute to hospitalisation, morbidity, and post-market drug withdrawal; identifying harmful combinations early is therefore a direct patient-safety concern.

The problem is fundamentally combinatorial. With thousands of approved drugs, possible combinations are too numerous for exhaustive clinical testing, and many interactions surface only after market release. Computational screening is consequently essential. Drugs, their protein targets, and protein interactions form a natural network, while a side effect between drugs can be represented as a typed edge. This project uses that perspective and the Decagon (BioSNAP) dataset as its biomedical knowledge graph.

1.2 Related Work

1.2.1 What is already solid

Multi-relational link prediction on a drug–protein graph is a well-established and effective formulation. Decagon [1] introduced it, while TIP [2] substantially improved efficiency, reporting AUPRC 0.890 (a 7.2% improvement over Decagon), about 83× faster execution, and roughly threefold lower memory use. Strong results have also been reported by molecular-structure approaches such as SSI-DDI [3] and biomedical-knowledge-graph approaches including KnowDDI [4] and MKG-FENN [5].

Most relevant here, a knowledge graph combined with a large language model (LLM) adds an interpretable explanation layer to prediction. DDI-GPT [6] retrieves a subgraph around a drug pair, converts it to text, and fine-tunes an LLM to return both a prediction and a gene-level rationale. More broadly, large-scale evaluation indicates that even modest LLMs can predict DDIs effectively [7]. Recent systems extend this direction with mechanism-conditioned multi-agent reasoning (DDIAgents [9]) and dynamic multi-view graph-and-language reasoning (DrugReason [10]).

1.2.2 What is still missing

Three gaps motivate this work. First, most systems, including DDI-GPT and DDIAgents, operate on drug pairs, whereas real polypharmacy commonly involves three or more medications. HODDI [8] explicitly targets high-order interactions, but is a benchmark with GNN and hypergraph baselines rather than an end-to-end multi-drug prediction-and-explanation system.

Second, locally hosted LLMs and chained multi-agent pipelines can be resource-intensive and add substantial inference latency. This project instead uses a single API-based LLM over a retrieved subgraph, prioritising simplicity and low latency while extending to multi-drug queries. Third, retrieval alone can expose only interactions already represented in the graph; it cannot readily

estimate calibrated probabilities for genuinely untested combinations. A trained graph model is therefore a future extension. Predictions also remain population-level and explanations are not necessarily clinician-ready.

1.3 Objective and Project Scope

1.3.1 Objective

Taking DDI-GPT as a baseline reference, this project develops an end-to-end decision-support prototype that screens and explains the side effects recorded for the drug pairs of a regimen of three or more drugs. It establishes an API-driven Graph-RAG pipeline over the Decagon biomedical graph. Instead of a heavy locally hosted pairwise model, a lightweight API-based LLM explains the retrieved evidence for a whole multi-drug regimen in a single request, in plain language and with low inference latency.

For a queried drug set, the system retrieves a local subgraph containing the drugs, their target proteins, protein–protein links among those targets, and known side-effect edges. It ranks the documented effects, serialises the evidence into compact context, and an API-based LLM writes a short natural-language explanation of the effects selected.

1.3.2 Dataset and suitability

The Decagon dataset from BioSNAP at Stanford is a standardised resource for polypharmacy side effects. It is provided as three connected CSV files:

- `bio-decagon-targets.csv`: human proteins targeted by 284 drugs (18,690 drug–protein edges).
- `bio-decagon-ppi.csv`: physical interactions among approximately 19,000 human proteins.
- `bio-decagon-combo.csv`: observed side effects, 1,317 types, for co-prescribed drug pairs.

In full, the graph contains 645 drugs, 19,089 proteins, 18,690 drug–protein edges, 715,612 protein–protein edges, and 4,649,441 drug–drug side-effect records (63,473 drug pairs). Only 284 of the 645 drugs have known targets. Standard identifiers connect the three files without manual labelling or external scraping, so entities and relations map directly to typed nodes and edges. This makes a compact local subgraph available for Graph-RAG.

1.3.3 Deliverable and boundaries

The deliverable is a functional, low-latency pipeline with a chat interface that accepts multi-drug queries, extracts a local subgraph, and returns ranked side effects with traceable evidence paths and short plain-language explanations. Training a graph neural network to predict novel unobserved adverse edges is outside the current phase. Continuous clinical variables—dose, pharmacokinetics, and renal/hepatic clearance—and wet-lab validation are also future work.

1.4 Report Structure

Table 1.1: Organisation of the report

Chapter	Content
1 — Introduction	Motivation, related work and gaps, objective and scope.
2 — Background	Knowledge graphs, Graph-RAG, LLMs for biomedical prediction, and the formal problem statement.
3 — Methodology	Multi-drug retrieval over the Decagon graph, pruning of noisy proteins, and tiered, traceable predictions.
4 — Experiments and Results	Data and implementation, evaluation of faithfulness and of known interactions, and limitations.
5 — Conclusion and Future Work	Outcomes and directions for extension.

Background

2.1 The Knowledge Graph

The biomedical knowledge graph contains two node types—drugs and proteins—and three relation types (Table 2.1). The drug–drug relation is sourced from Decagon.

Table 2.1: Relation types in the knowledge graph

Relation	Description
Drug $\rightarrow$ Protein	Proteins a drug binds to or modulates. This is a property of an individual drug and does not depend on co-prescribed drugs.
Protein $\leftrightarrow$ Protein	A fixed, drug-independent network of physical protein interactions, including signal transduction and shared pathways.
Drug $\leftrightarrow$ Drug	Previously documented side effects for specific drug pairs, as recorded in Decagon.

Only the third relation is inherently pairwise: each polypharmacy edge connects exactly two drugs. Drug–target and protein–protein relations remain available for any number of drugs.

2.2 Standard Graph-RAG

Graph-RAG has two stages. During retrieval, the graph is traversed outward from entities named in a query to collect a relevant neighbourhood, preserving relational structure rather than relying only on textual similarity. During generation, that neighbourhood is serialised as text and supplied to an LLM, allowing the model to reason over explicit structure rather than memorised associations. Retrieval is not intrinsically limited to two entities because it can expand from any query set.

2.3 Problem Statement

Given a drug regimen of $k \geq 3$ drugs, the task is to predict potential side effects. Each prediction is annotated with an interpretive tier and the supporting path in the retrieved subgraph.

Methodology

The pipeline operates on the Decagon knowledge graph. Figure 3.1 illustrates the data structure: drug and protein nodes are connected by drug–protein, protein–protein, and pairwise drug–drug side-effect edges. Retrieval traverses the first two edge types for every drug in a queried regimen, while drug–drug edges supply known pairwise history.

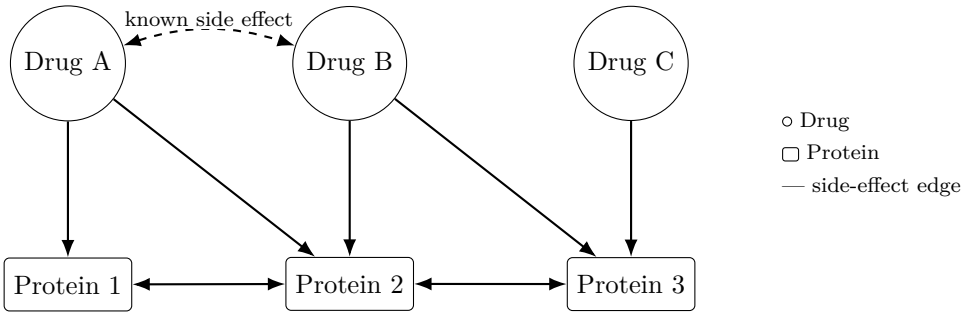


Figure 3.1: Example Decagon knowledge graph and the relation types used for retrieval.

The proposed pipeline (Figure 3.2) has three stages: retrieval of the queried drugs' neighbourhood, pruning of structurally noisy proteins, and generation of tiered, traceable predictions.

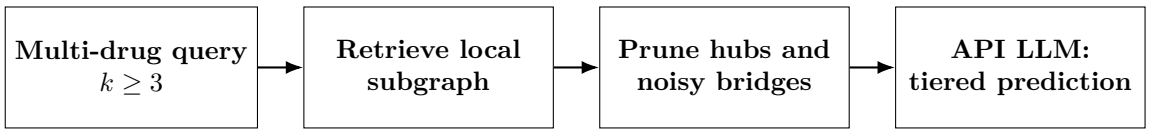


Figure 3.2: Overview of the proposed Graph-RAG pipeline for a regimen of $k \geq 3$ drugs.

3.1 Extending Retrieval from Two to $k \geq 3$ Drugs

Retrieval over drug–target and protein–protein relations is conducted independently for each drug, then combined; it therefore generalises to any k without modification (Table 3.1).

Table 3.1: Retrieval sequence for a regimen of $k \geq 3$ drugs

Step	Operation	Description
1	Look up each drug's targets	For every drug in the regimen, retrieve its target-protein set independently.
2	Combine target sets	Take the union of all targets to form one neighbourhood.
3	Find protein bridges	Find direct target-to-target protein edges or paths through one intermediate protein.
4	Find shared targets	Identify proteins targeted by three or more drugs by intersecting the target sets.
5	Attach pairwise history	For every drug pair, add documented side effects so the LLM knows existing evidence.
6	Serialise	Resolve identifiers to standard drug and gene names; convert drugs, targets, bridges, shared targets, and history to text for the LLM.

Bridges are restricted to at most two hops. Longer paths would introduce many indirectly related proteins and make the neighbourhood noisy; the restriction keeps each bridge biologically direct. Shared targets expose a signal unavailable to pairwise analysis: a protein targeted by three drugs becomes visible only after their neighbourhoods are combined. Proteins targeted by exactly two drugs remain in the neighbourhood and are covered by pairwise history. The assembled context is passed to the LLM, which only explains the effects already selected and ranked (Section 4.1.4).

3.2 Pruning Before Passing to the LLM

Housekeeping proteins can interact with very many proteins and flood a neighbourhood with mechanism-irrelevant information. They are filtered before serialisation. A direct target of any regimen drug is always retained; other proteins are retained only if their global protein-network connectivity is below a chosen threshold. A protein above that threshold is treated as a generic hub and removed together with bridges that depend on it.

3.3 Output: Tiered, Traceable Predictions

The LLM output is organised into three tiers according to the evidence supporting a prediction (Table 3.2).

Table 3.2: Interpretive tiers of predicted side effects

Tier	Trigger	Clinical meaning
1: Known pairwise effect	A documented side effect exists for one drug pair in the regimen.	An independent effect of that pair, unrelated to the other drug(s).
2: Amplified effect	A protein is a shared target of three or more drugs, and a pairwise effect is documented for at least one pair among them.	A documented effect may be escalated because several drugs act on the same target.
3: Hypothesised effect	A bridge links targets of two different drugs, and that pair has no documented side effect at all.	A hypothesis for an untested pair, based only on protein links; it requires clinical validation.

The model is constrained to a fixed structured format (for example, JSON). For every predicted side effect it must state the assigned tier, a supporting subgraph path, and a plain-language explanation. Each cited path is automatically checked against the retrieved subgraph; a prediction with no valid path is discarded. Tier 2 and Tier 3 claims are thus traceable to retrieved structure and machine-readable for downstream use. Tier 3 predictions remain hypotheses requiring experimental or clinical validation.

Experiments and Results

4.1 Implementation

This section describes how the Graph-RAG pipeline of Chapter 3 is realised as a working system: the data (Section 4.1.1), the software architecture (4.1.2), retrieval and ranking (4.1.3), the use of the large language model (LLM) (4.1.4), and the validation of its output (4.1.5).

4.1.1 Data

The knowledge graph is built from the Decagon dataset [1], which the Stanford SNAP BioSNAP collection distributes as four CSV files; their sizes are given in Section 1.3.2. The fourth file, which lists single-drug side effects, is optional and serves only as a ranking feature (Section 4.1.3), never for retrieval.

Two structural properties of the graph shaped the implementation. First, both the drug–target and the protein–protein subgraphs are heavily right-skewed (Table 4.1): half of all drugs bind at most 11 proteins, yet one binds 2,840, and half of all proteins have at most 33 interaction partners, yet the busiest has 2,519. Letting such hub proteins act as bridges would connect almost any two drugs through an uninformative path. Setting `hub_degree_threshold` to 500 (Table 4.3) removes only the extreme tail, 220 of 19,081 proteins (1.15%), close to the 99th percentile.

Table 4.1: Degree-distribution statistics of the drug–target and protein–protein graphs

Degree	Min	Median	Mean	P90	P99	Max
Targets per drug	1	11	65.8	170	206	2,840
Drugs per target protein	1	1	5.1	5	78	94
PPI neighbours per protein	1	33	75.0	201	525	2,519

Second, the 645 drugs with documented pairwise history in `bio-decagon-combo.csv` are not the same population as the 284 drugs in `bio-decagon-targets.csv`: only 284 of 645 (44%) have a known target set. For the other 361, Tier 1 evidence (Table 3.2) is available whenever the pair was tested, but Tier 2 and Tier 3 need a protein-level path and are unreachable.

Third, the documented history is heavy-tailed at two levels (Table 4.2). A drug pair with any documented effect has a median of 53 and up to 540, which is why effects are ranked (Eq. (4.1)) rather than listed; and an effect code is documented for a median of 1,596 pairs but for as few as one, which is why the inverse-frequency term of Eq. (4.2) matters. Of the 1,317 effect codes in the raw file, 963 occur in at least 500 pairs, matching within one the 964 effects that Zitnik et al.

[1] evaluate after the same cut-off; the remaining ~350 receive a larger $I(e)$ rather than being dropped.

Table 4.2: Side-effect frequency statistics in bio-decagon-combo.csv

Quantity	Min	Median	Mean	P90	P99	Max
Effects per documented pair	1	53	73.3	165	305	540
Pairs documenting an effect	1	1,596	3,530.3	9,530	21,396	28,568

At run time, drug–target and protein–protein edges are loaded once into in-memory adjacency maps for constant-time neighbour look-ups. The 4,649,441-row combination file is materialised once into a local SQLite table `side_effects` keyed by (drug pair, effect code) with an index on the second drug, plus two derived tables used by the ranking formula: `effect_counts` (n_e, pairs documenting each effect) and `stats` (N, distinct documented pairs). A SHA-256 signature of each source file and a schema version are stored with it, so the cache is rebuilt only when a source file or the schema changes.

Free text typed by a user, including Vietnamese brand names, typos, or several drugs in one sentence, is resolved to a Decagon STITCH identifier by a rule-based resolver: (i) a STITCH-ID pattern, (ii) an exact match on normalised generic names, synonyms, or Vietnamese brand aliases, then (iii) fuzzy string matching (RapidFuzz), accepted automatically above a score of 90 and offered as a suggestion above 75. A curated stop-word list removes conversational filler first, so that *"em đang uống panadol với ibuprofen"* resolves to the two intended drugs. Generic names, brand aliases, gene symbols, Vietnamese effect names and the EMA seriousness lists come from resource tables built offline.

4.1.2 Software Architecture

The system is written in Python. `app.py` is a Streamlit front-end that calls the single entry point `PolypharmacySystem.predict()` of `pipeline.py`. The package `graphrag/` holds `config.py` (settings and ranking weights), `dataset.py` (loading, SQLite cache, subgraph retrieval), `names.py` and `resolver.py` (name resolution), `ranking.py` (effect scoring), `llm.py` (prompt, API call, validation), `presenter.py` (assembling results) and `visualize.py` (interaction graph). Paths, the LLM endpoint and model, and the parameters of Table 4.3 are read from a `.env` file, so they can change without editing code. Dependencies are kept small: `httpx` for the LLM call, `pydantic` for output validation, `rapidfuzz` for name resolution and `pyvis` for the interaction graph.

4.1.3 Subgraph Retrieval, Pruning, and Ranking

Retrieval follows the six steps of Table 3.1 within the bounds of Table 4.3. A protein is pruned as a hub once its PPI degree exceeds `hub_degree_threshold`: it is never used as a bridge intermediate, although a direct target of a queried drug stays an endpoint. Bridges are searched only for pairs with no recorded effect at all, are bounded per pair, and are visited in a fixed order, so results are reproducible and truncation is flagged. The evidence sent to the LLM is capped at `max_evidence` and shared across drug pairs round-robin, so one pair cannot use the whole budget. Documented (Tier 1 and 2) evidence is built only for the effects that will be shown, and Tier 2 and Tier 3 each have a reserved quarter of the budget, whatever a tier leaves unused going to the others. Without this reservation Tier 1 alone filled the budget in larger regimens: in 40 six-drug regimens built around a common target, no Tier 2 or Tier 3 evidence

reached the LLM; with it, every one of them carries both. Paths through pruned hubs are removed before the budget is spent.

Table 4.3: Retrieval and pruning bounds (default values)

Parameter	Default	Role
`hub_degree_threshold`	500	PPI degree above which a protein is pruned
`max_evidence`	400	Cap on evidence items serialised to the LLM
`tier3_one_hop_per_pair`	8	One-hop bridges sampled per undocumented pair
`tier3_two_hop_per_pair`	4	Two-hop bridges sampled per undocumented pair
`tier3_candidates_per_pair`	20	Candidate labels offered for a Tier 3 pair
`max_serious_effects`	10	Serious effects surfaced per analysis
`max_other_effects`	5	Other effects surfaced per analysis
`hub_degree_percentile`	0 (off)	Hub threshold as a PPI degree percentile
`convergence_amplification`	off	Count one-hop target convergence as Tier 2

Because a pair can list dozens to hundreds of effects, every documented effect e in the regimen receives a transparent additive score

$$\text{score}(e) = w_r R(e) + w_i I(e) + w_m M(e) + w_s S(e) + w_c S(e) C(e) + w_a A(e), \qquad (4.1)$$

where $R(e)$ is the number of regimen pairs documenting e; $M(e)$, $S(e)$ and $C(e)$ are binary indicators for "absent from both drugs' individual side-effect lists", "an EMA Important Medical Event", and "additionally life-threatening"; $A(e)$ indicates that a pair documenting e also shares a protein target with a third drug (the Tier 2 trigger); and $I(e)$ is an inverse pair-frequency term,

$$I(e) = \log\,(\ N\ /\ \max(n_e,\ 1)\ ), \qquad (4.2)$$

with N the number of distinct documented pairs and n_e the number documenting e, so effects specific to few pairs outweigh ubiquitous ones. The weights are $w_r = 1.0$, $w_i = 0.5$, $w_m = 1.0$, $w_s = 2.0$, $w_c = 2.0$, $w_a = 1.0$, set by hand and exposed as configuration for ablation. In Decagon, $M(e)$ equals 1 for every effect (4,020 of 4,020 effects in 40 random regimens), because polypharmacy effects are by definition not attributed to single drugs; it therefore adds a constant and does not change the order.

4.1.4 Large Language Model Usage

The LLM is reached through OpenRouter's OpenAI-compatible `/chat/completions` endpoint, so the model can be swapped through `LLM_MODEL` without code changes; the default is Google's `gemini-2.5-flash`. It is used as a constrained explanation layer over evidence that was already retrieved, ranked and bounded. Each request asks for `json_object` output with the model's extended reasoning disabled, to keep latency and cost low, and uses `max_tokens` = 4000, a 120 s timeout (15 s to connect) and up to three tries with back-off on connection failure only. The content of each prediction is fixed by the serialised evidence and checked afterwards.

The system prompt fixes the output contract: a JSON object with a `predictions` array whose items carry `side_effect`, `tier`, one or more `evidence_ids`, and a Vietnamese `explanation` of one or two short sentences describing the side effect in plain words (the interface lists the drugs). It restates the tier semantics of Table 3.2 as hard constraints: Tier 1 must reproduce a documented pairwise effect; Tier 2 additionally requires a target shared by at least three drugs; Tier 3 is a hypothesis, allowed only when a retrieved bridge links an undocumented pair, with its label copied verbatim from the bridge's candidates. Output is capped at 15

predictions (Tier 1 first, at most two Tier 3), and the model must return an empty array rather than invent evidence.

4.1.5 Grounding, Validation, and Fallback Behaviour

The response is parsed against a strict Pydantic schema; if the model wrapped the JSON in a code fence or stopped mid-stream, a tolerant path keeps every complete, schema-valid object found before the cut-off instead of discarding the response. Parsing success does not make a prediction trustworthy, so each item is re-validated against the retrieved subgraph: every cited evidence identifier must exist and carry the declared tier; for Tiers 1 and 2 the label must match every cited item, and for Tier 3 it must be among the bridge's candidate labels; each cited edge is checked against the target, PPI and side-effect tables; duplicates are removed. A Tier 2 item that fails is downgraded to Tier 1 when the evidence it cited already includes documented Tier 1 items for the same label; nothing new is added at this step. Any other failing item is dropped and logged with its reason (unknown identifier, tier mismatch, invalid path, label mismatch), so the rate of fabricated evidence can be audited. This check covers evidence and labels, not the wording of the explanation.

If the LLM call fails (no API key, non-conforming JSON, HTTP or network error), the pipeline still returns the ranked evidence and coverage summary, with template explanations and a diagnostic message.

4.2 Evaluation

4.2.1 What can and cannot be measured

Decagon records only whether a side effect was reported for a pair of drugs. It has no ground truth for regimens of three or more drugs, so no accuracy figure for a whole regimen can be computed. The evaluation therefore asks three narrower questions: whether every result can be traced to the source data (Section 4.2.2); whether well-established dangerous interactions stay visible as the regimen grows (4.2.3); and how far the Tier 3 hypotheses are supported (4.2.4). All numbers come from scripts in the repository, with fixed seeds and without calling the LLM.

4.2.2 Faithfulness to the data

For 200 random regimens of 3–6 drugs, every returned effect and evidence path was checked against the original CSV files, read independently of the SQLite cache (Table 4.4). The output matches the data exactly; this shows that the system reproduces what is recorded, not that the recorded effects occur in a patient taking the regimen. The output check of Section 4.1.5 was tested on 1,250 predictions, 300 valid and 950 deliberately corrupted (invented evidence identifiers, wrong tiers, wrong labels): all valid ones were kept and all corrupted ones rejected. The name-recognition tests use inputs written by the authors, so they do not show performance on arbitrary users.

4.2.3 Known interactions in larger regimens

The benchmark has 57 well-known pairwise interactions (for example warfarin with an NSAID, the NSAID, ACE inhibitor and diuretic "triple whammy", an opioid with a benzodiazepine), each with its expected clinical effect, chosen from standard pharmacological knowledge and not reviewed by a clinician; two contain a drug absent from Decagon and are skipped, leaving 55.

Table 4.4: Faithfulness and robustness checks

Check	Passed / total	Rate
Tier 1 effects returned are in the combo CSV (precision)	52,880 / 52,880	100%
Tier 1 effects of the returned pairs are all present (recall)	52,880 / 52,880	100%
Tier 2 pairs satisfy the shared-target rule	72 / 72	100%
Evidence paths correct edge by edge (Tiers 1–3)	40,529 / 40,529	100%
Corrupted LLM predictions rejected	950 / 950	100%
Valid LLM predictions kept	300 / 300	100%
Drug names recognised, 71 typed inputs (none wrong, 1 asked back)	70 / 71	98.6%
Chat sentences with the right drugs, 30 sentences	30 / 30	100%

Each is a three-drug regimen with one interacting pair and a bystander. A hit means the expected effect is among the first 3 or 10 entries of the serious-effect list (the interface shows 10). To test larger regimens, each case was extended with 1, 2, 3 or 5 random bystander drugs from the other cases (five draws per case and size). Separately, 20 regimens of 4–8 drugs were written by hand, each with one known interacting pair; for two, Decagon has no record of the expected effect for the pair, so they cannot be scored and are left out (18 remain). Table 4.5 gives the results; chance is the probability that a random ordering of the serious effects puts the expected one in the first 10.

Table 4.5: Known interaction recovered, by regimen size

Regimens	Runs	Hit@3	Hit@10	Chance@10	MRR
3 drugs (benchmark)	55	23.6%	52.7%	11.7%	0.208
4 drugs (+1 bystander)	275	18.2%	47.6%	8.4%	0.189
5 drugs (+2)	275	12.7%	44.4%	6.7%	0.149
6 drugs (+3)	275	14.5%	43.3%	5.8%	0.153
8 drugs (+5)	275	11.6%	40.0%	4.9%	0.150
4–8 drugs, hand-written	18	11.1%	27.8%	6.4%	0.134

The signal weakens as drugs are added, from 52.7% at three drugs to 40.0% at eight (hit@3: 23.6% to 11.6%), and is lower still in the hand-written regimens (5 of 18 in the top 10), but stays several times above chance. Two reasons explain the decline: the data record presence only, so one specific interaction competes with hundreds of other documented effects, and each added drug brings its own pairs. With 18 cases, one or two cases move a rate by 5 to 10 points, so the rows show a common trend, not precise estimates. In a regimen of four or more drugs the interface, which shows 10 serious effects, will often not display the effect of a known dangerous pair.

4.2.4 Tier 3 hypotheses

Tier 3 rests on protein bridges. Used alone to decide whether a pair of drugs has any documented interaction, protein links between targets give an AUROC of 0.54, close to chance (0.50), while the popularity of the drugs alone gives 0.94, so bridges are not evidence that a pair interacts. For 2,000 pairs with their data hidden, the label suggested for Tier 3 has precision@10 of 0.507, against 0.418 for the globally most common effects and 0.065 for random labels; the per-effect ranking test shows the same pattern (AUROC 0.951 against 0.908 for popularity). Much of what

the data support reflects how often a drug is reported, not a mechanism, so Tier 3 is a hypothesis for review, never a finding.

4.3 Limitations

Data. Only the 645 Decagon drugs are covered (others are reported as absent, not guessed), and only 284 have known targets, so the other 361 yield Tier 1 only. Only 30.6% of the 207,690 possible pairs have a recorded effect, and a missing pair is not a safe pair. The data hold no dose, frequency, severity, age or comorbidity, and come from spontaneous reports with reporting bias and noise.

Three or more drugs. All evidence is pairwise: a regimen of k drugs is treated as its $k(k-1)/2$ pairs, ranked and filtered. The system screens for dangerous pairs inside a regimen; it does not identify effects that arise only when three or more drugs are taken together. Tier 2 marks a shared target, a structural clue not shown to amplify an effect: the target table records no direction of action (agonist, antagonist, inhibitor), and two drugs at one receptor can as well oppose each other. Widening Tier 2 to proteins within one PPI hop of three drugs' targets (an option, off by default) moved hit@10 by under two points either way on the benchmarks of Table 4.5, within the noise of so few cases, and setting the Tier 2 ranking weight to zero left the three-drug hit@10 unchanged. Metabolic interactions (CYP enzymes, transporters, QT prolongation), which cause many dangerous combinations, are not represented as such.

Accuracy. The strongest results (Table 4.4) measure agreement with the data, not clinical correctness. The clinical tests are small, chosen by the authors and list one expected effect per case: about half of the known interactions reach the top 10 in three-drug regimens, but 5 of 18 (27.8%) in hand-written regimens of four to eight drugs, so the displayed list often misses a known dangerous pair in larger regimens. The ranking weights were not tuned on these cases.

Input and language model. Drug names are recognised by rules, not by a learned model: drug classes ("anti-inflammatory"), negation ("allergic to aspirin") and unusual typos are not handled, and unrelated words are sometimes proposed as drugs for the user to reject. The language model only writes a short general description of each side effect, which comes from its general knowledge rather than from the data, was not evaluated, and is labelled as AI-written in the interface. The output is population-level information, not medical advice.

Conclusion and Future Work

This report presented a Graph-RAG pipeline that takes a regimen of three or more drugs, retrieves the pairwise side-effect history and protein evidence for it from the Decagon graph, ranks the effects with a transparent score, and asks a single API-based LLM to describe them, with every cited item re-checked against the data.

The results support a modest conclusion. The pipeline reproduces the data exactly and rejects unsupported model output (Table 4.4), and it ranks known dangerous pairs well above chance, though less so as regimens grow: 52.7% in the top 10 for three drugs and 27.8% for hand-written regimens of four to eight (Table 4.5). The protein-based Tier 2 and Tier 3 evidence adds little beyond how often a drug is reported. The system is therefore a screening and explanation aid over recorded pairwise reports, not a predictor of new interactions.

Future work follows from these limits: review of the benchmark by a clinician and a larger set of high-order regimens, for example from HODDI [8]; frequency information from the underlying spontaneous reports, to replace presence-only ranking with a proper disproportionality score; a trained graph model for calibrated probabilities on untested combinations (Section 1.2.2); a per-pair view for large regimens, so that the top-10 cut-off cannot hide a known dangerous pair; and recognition of drug classes and negation in the input.

Bibliography

[1] M. Zitnik, M. Agrawal, and J. Leskovec, "Modeling polypharmacy side effects with graph convolutional networks (Decagon)," *Bioinformatics*, vol. 34, no. 13, pp. i457–i466, 2018.

[2] H. Xu, S. Sang, and H. Lu, "Tri-graph information propagation for polypharmacy side effect prediction (TIP)," *arXiv:2001.10516*, 2020.

[3] A. K. Nyamabo, H. Yu, and J.-Y. Shi, "SSI-DDI: substructure–substructure interactions for drug–drug interaction prediction," *Briefings in Bioinformatics*, vol. 22, no. 6, bbab133, 2021.

[4] Y. Wang, Z. Yang, and Q. Yao, "Accurate and interpretable drug–drug interaction prediction enabled by knowledge subgraph learning (KnowDDI)," *Communications Medicine*, 2024.

[5] D. Wu, W. Sun, Y. He, Z. Chen, and X. Luo, "MKG-FENN: A multimodal knowledge graph fused end-to-end neural network for DDI prediction," *AAAI*, vol. 38, no. 9, pp. 10216–10224, 2024.

[6] C. Xu, K. C. Bulusu, H. Pan, and O. Elemento, "DDI-GPT: Explainable prediction of drug–drug interactions using LLMs enhanced with knowledge graphs," *bioRxiv*, 2024.

[7] G. De Vito, F. Ferrucci, and A. Angelakis, "LLMs for drug–drug interaction prediction: A comprehensive comparison," *arXiv:2502.06890*, 2025.

[8] Z. Wang et al., "HODDI: A dataset of high-order drug–drug interactions for computational pharmacovigilance," *arXiv:2502.06274*, 2025.

[9] Z. Shen, Y. Liu, Q. Yao, et al., "DDIAgents: Mechanism-conditioned context flow for drug–drug interaction prediction," *arXiv:2606.31085*, 2026.

[10] Z. Liu et al., "DrugReason: Dynamic multi-view reasoning over knowledge graph and language evidence," *arXiv:2609.06779*, 2026.